

Bài thực hành số 3: MongoDB

Cho một tập các lược đồ quan hệ biểu diễn một CSDL quản lý việc sản xuất các mặt hàng máy in như sau:

NSX (MaNSX, TenNSX, DiaChi)

SP (MaNSX, MaSP, loai)

PC (MaSP-P, CPU, RAM, HD, gia)

Laptop (MaSP-L, CPU, RAM, HD, ManHinh, gia)

A) Hãy chuyển đổi cơ sở dữ liệu này sang dạng hệ cơ sở dữ liệu NoSQL (MongoDB)?

Hướng dẫn:

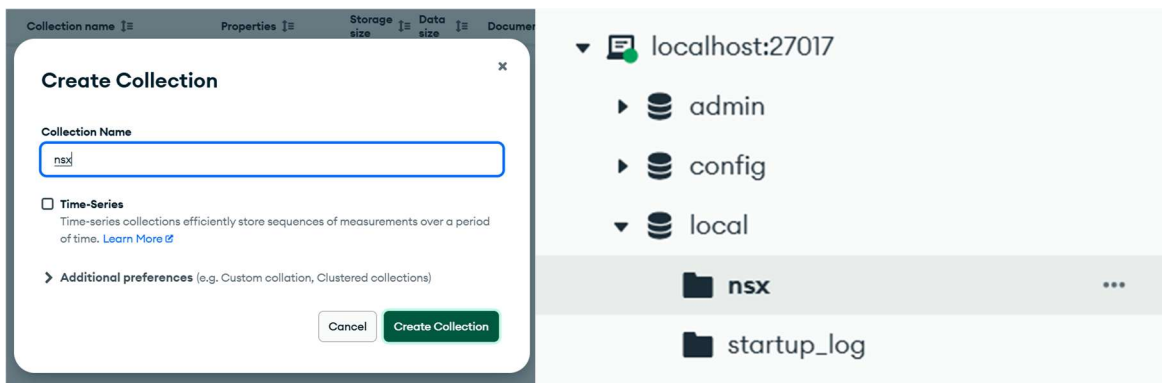
1) Lưu ý: Cần điều chỉnh cách thiết kế dữ liệu để tận dụng schema linh hoạt và không cần join của MongoDB.

2) Thiết kế MongoDB tổng thể: có 2 cách:

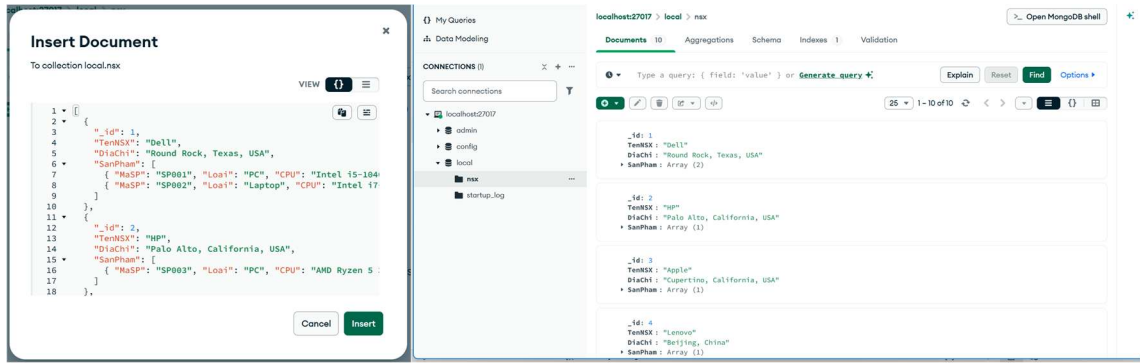
- Mô hình nhúng: Có thể gom dữ liệu vào 1 collection chính: nsx (Nhà sản xuất), nhúng thông tin sản phẩm vào bên trong.

=> sau này khi truy vấn không cần join giữa NSX và SP, PC, Laptop; để truy xuất tất cả các sản phẩm của một NSX

=> Khi số lượng sản phẩm của mỗi NSX không quá lớn, cần truy vấn nhanh theo NSX



Tạo 1 collection chính

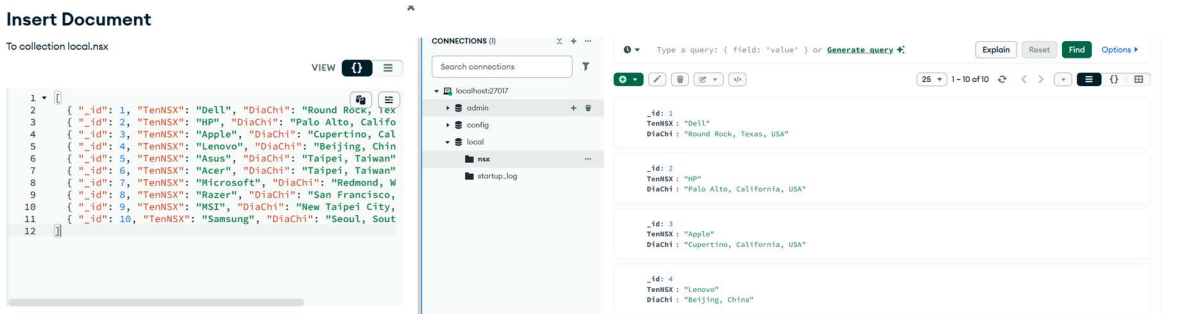


Ảnh minh họa cách thiết kế MongoDB theo mô hình nhúng, trong đó thông tin sản phẩm (SP, PC, Laptop) được lưu trực tiếp bên trong document của NSX. Cách này giúp truy vấn nhanh khi cần lấy toàn bộ sản phẩm của một nhà sản xuất mà không cần JOIN. Tuy nhiên, nhược điểm là document có thể trở nên rất lớn và dễ vượt giới hạn 16MB nếu số lượng sản phẩm nhiều.

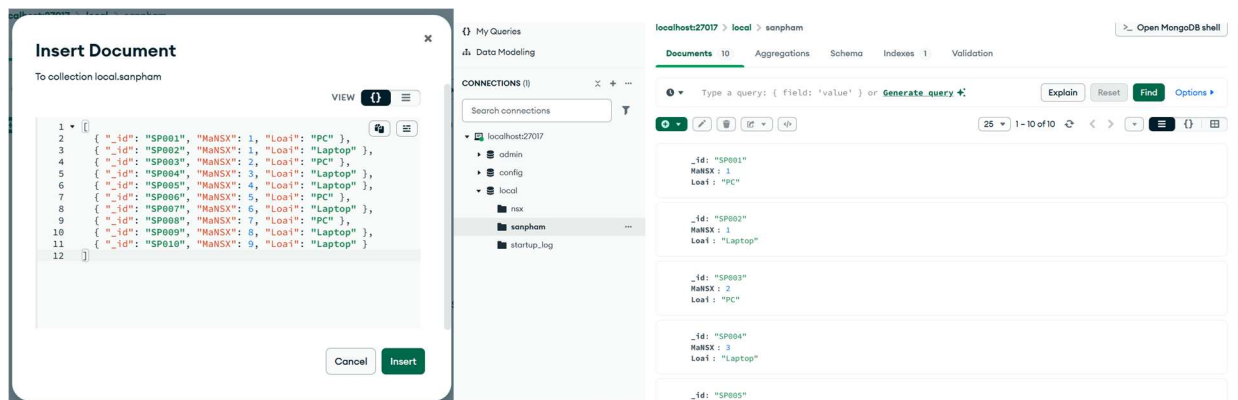
- Mô hình tách nhiều collection: nsx, sanpham, pc, labtop.

=> gần giống mô hình quan hệ, dễ chuyển đổi; Có thể dùng nếu số lượng sản phẩm lớn và truy xuất riêng biệt; Khó khăn: cần xử lý logic để "join" bằng ứng dụng (do MongoDB không có JOIN thật sự).

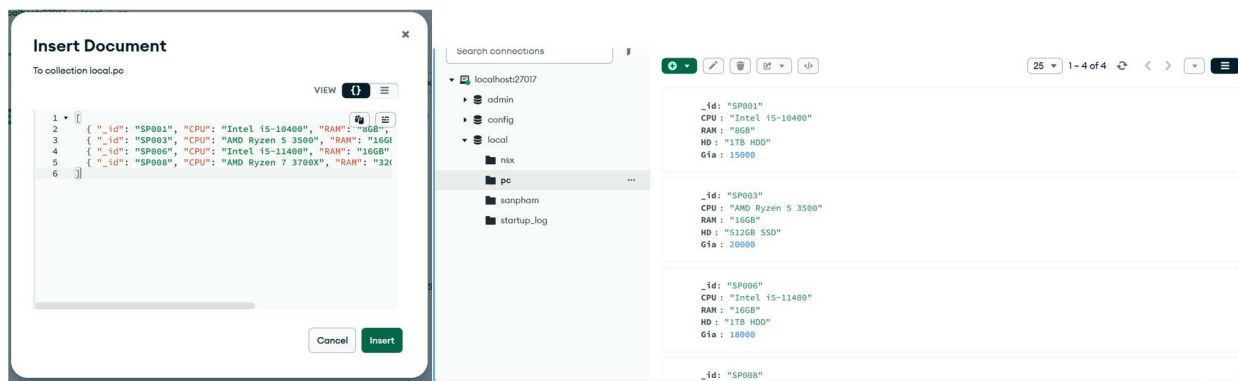
=> Khi số lượng sản phẩm rất lớn hoặc cần truy vấn riêng sản phẩm mà không cần NSX



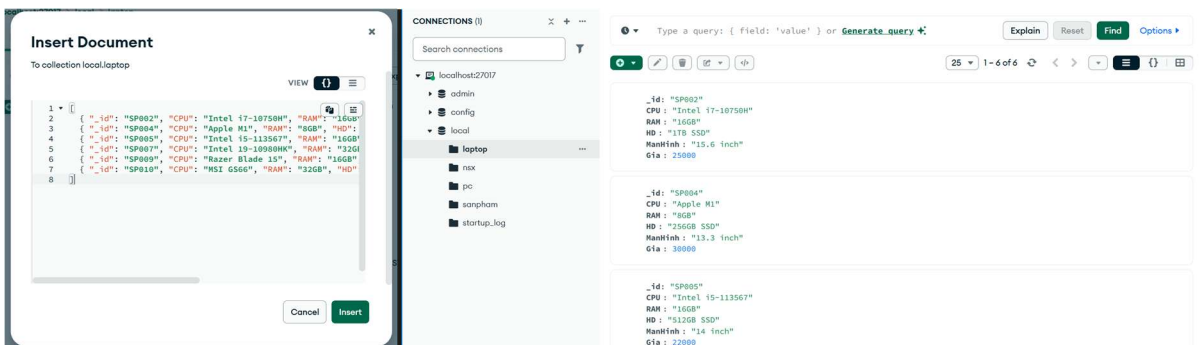
Nhúng dữ liệu cho collection nsx



Những dữ liệu cho collection sanpham



Những dữ liệu cho collection pc



Những dữ liệu cho collection laptop

Phân tích cách tối ưu: Đối với bài toán quản lý mặt hàng máy tính, **Cách 2 (Mô hình tách collection)** là phương án tối ưu nhất. Lý do là trong thực tế, số lượng sản phẩm của các hãng rất lớn và người dùng thường xuyên có nhu cầu tìm kiếm, lọc sản phẩm độc lập theo cấu hình (ví dụ: tìm mọi Laptop có RAM 16GB, giá dưới 30 triệu) thay vì tìm theo nhà sản xuất. Việc phân tách collection sẽ giúp hệ thống tránh được lỗi tràn dung lượng bộ nhớ (giới hạn 16MB cho mỗi document của MongoDB), đồng thời giúp việc truy vấn, lập chỉ mục và phân trang sản phẩm diễn ra nhanh chóng, dễ dàng mở rộng quy mô về lâu dài.

B) Hãy triển khai cơ sở dữ liệu đã chuyển đổi sang MongoDB

1) Tạo database và collection

Create Database



TH3

Collection Name

☐ Time-Series

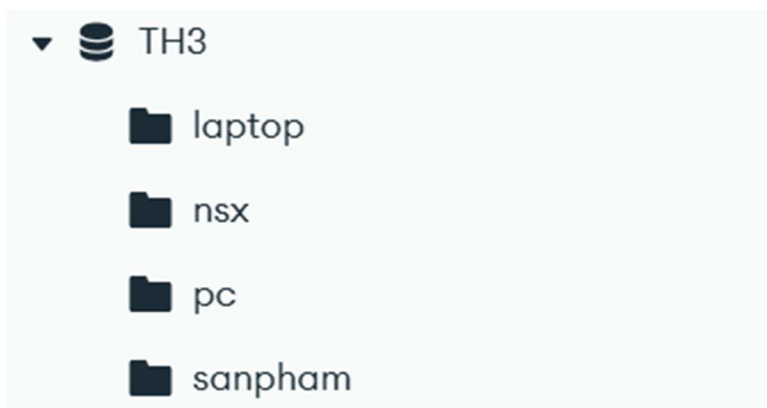
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Clustered collections)

Tạo Database

```
TH3> db.createCollection("nsx")
| db.createCollection("sanpham")
| db.createCollection("pc")
| db.createCollection("laptop")
{ ok: 1 }
```

Query tạo collection



Kết quả sau khi tạo

2) Thêm dữ liệu mẫu (insert documents)

```
TH3> db.nsx.insertMany([
  { "_id": 1, "TenNSX": "Dell", "DiaChi": "Round Rock, Texas, USA" },
  { "_id": 2, "TenNSX": "HP", "DiaChi": "Palo Alto, California, USA" },
  { "_id": 3, "TenNSX": "Apple", "DiaChi": "Cupertino, California, USA" },
  { "_id": 4, "TenNSX": "Lenovo", "DiaChi": "Beijing, China" },
  { "_id": 5, "TenNSX": "Asus", "DiaChi": "Taipei, Taiwan" },
  { "_id": 6, "TenNSX": "Acer", "DiaChi": "Taipei, Taiwan" },
  { "_id": 7, "TenNSX": "Microsoft", "DiaChi": "Redmond, Washington, USA" },
  { "_id": 8, "TenNSX": "Razer", "DiaChi": "San Francisco, California, USA" },
  { "_id": 9, "TenNSX": "MSI", "DiaChi": "New Taipei City, Taiwan" },
  { "_id": 10, "TenNSX": "Samsung", "DiaChi": "Seoul, South Korea" }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

Query thêm dữ liệu nsx

```
TH3> db.sanpham.insertMany([
  { "_id": "SP001", "MaNSX": 1, "Loai": "PC" },
  { "_id": "SP002", "MaNSX": 1, "Loai": "Laptop" },
  { "_id": "SP003", "MaNSX": 2, "Loai": "PC" },
  { "_id": "SP004", "MaNSX": 3, "Loai": "Laptop" },
  { "_id": "SP005", "MaNSX": 4, "Loai": "Laptop" },
  { "_id": "SP006", "MaNSX": 5, "Loai": "PC" },
  { "_id": "SP007", "MaNSX": 6, "Loai": "Laptop" },
  { "_id": "SP008", "MaNSX": 7, "Loai": "PC" },
  { "_id": "SP009", "MaNSX": 8, "Loai": "Laptop" },
  { "_id": "SP010", "MaNSX": 9, "Loai": "Laptop" }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': 'SP001',
    '1': 'SP002',
    '2': 'SP003',
    '3': 'SP004',
    '4': 'SP005',
    '5': 'SP006',
    '6': 'SP007',
    '7': 'SP008',
    '8': 'SP009',
    '9': 'SP010'
  }
}
TH3>
```

Query thêm dữ liệu sanpham

```

TH3> db.pc.insertMany([
|   { "_id": "SP001", "CPU": "Intel i5-10400", "RAM": "8GB", "HD": "1TB HDD", "Gia": 15000.00 },
|   { "_id": "SP003", "CPU": "AMD Ryzen 5 3500", "RAM": "16GB", "HD": "512GB SSD", "Gia": 20000.00 },
|   { "_id": "SP006", "CPU": "Intel i5-11400", "RAM": "16GB", "HD": "1TB HDD", "Gia": 18000.00 },
|   { "_id": "SP008", "CPU": "AMD Ryzen 7 3700X", "RAM": "32GB", "HD": "2TB SSD", "Gia": 25000.00 }
| ]);
| {
|   acknowledged: true,
|   insertedIds: { '0': 'SP001', '1': 'SP003', '2': 'SP006', '3': 'SP008' }
| }
TH3> |

```

Query thêm dữ liệu cho pc

```

TH3> db.laptop.insertMany([
|   { "_id": "SP002", "CPU": "Intel i7-10750H", "RAM": "16GB", "HD": "1TB SSD", "ManHinh": "15.6 inch", "Gia": 25000.00
| },
|   { "_id": "SP004", "CPU": "Apple M1", "RAM": "8GB", "HD": "256GB SSD", "ManHinh": "13.3 inch", "Gia": 30000.00 },
|   { "_id": "SP005", "CPU": "Intel i5-113567", "RAM": "16GB", "HD": "512GB SSD", "ManHinh": "14 inch", "Gia": 22000.00
| },
|   { "_id": "SP007", "CPU": "Intel 19-10980HK", "RAM": "32GB", "HD": "1TB SSD", "ManHinh": "15.6 inch", "Gia": 35000.00
| },
|   { "_id": "SP009", "CPU": "Razer Blade 15", "RAM": "16GB", "HD": "512GB SSD", "ManHinh": "15.6 inch", "Gia": 28000.00
| },
|   { "_id": "SP010", "CPU": "MSI GS66", "RAM": "32GB", "HD": "1TB SSD", "ManHinh": "15.6 inch", "Gia": 33000.00 }
| ]);
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': 'SP002',
|     '1': 'SP004',
|     '2': 'SP005',
|     '3': 'SP007',
|     '4': 'SP009',
|     '5': 'SP010'
|   }
| }
TH3> |

```

Query thêm dữ liệu cho laptop

3) Truy vấn ví dụ (MongoDB queries)

- Tìm tất cả sản phẩm của NSX có mã "NSX01"

```

TH3> let q1 = db.sanpham.find({ MaNSX: 1 }).toArray();
| printjson(q1);
| [
|   {
|     _id: 'SP001',
|     MaNSX: 1,
|     Loai: 'PC'
|   },
|   {
|     _id: 'SP002',
|     MaNSX: 1,
|     Loai: 'Laptop'
|   }
| ]
TH3> |

```

- Lấy tên NSX và giá các sản phẩm PC

```

TH3> let q2 = db.pc.aggregate([
  {
    // Nối bảng pc với bảng sanpham để lấy mã nhà sản xuất
    $lookup: {
      from: "sanpham",
      localField: "_id",
      foreignField: "_id",
      as: "ThôngTinChung"
    }
  },
  { $unwind: "$ThôngTinChung" },
  {
    // Nối tiếp với bảng nsx để lấy tên
    $lookup: {
      from: "nsx",
      localField: "ThôngTinChung.MaNSX",
      foreignField: "_id",
      as: "ThôngTinNSX"
    }
  },
  { $unwind: "$ThôngTinNSX" },
  {
    // Chỉ chọn ra các cột cần hiển thị
    $project: {
      "_id": 0,
      "MaSP": "$_id",
      "TenNSX": "$ThôngTinNSX.TenNSX",
      "Gia": "$Gia"
    }
  }
]).toArray();
printjson(q2);

{
  MaSP: 'SP001',
  TenNSX: 'Dell',
  Gia: 15000
},
{
  MaSP: 'SP003',
  TenNSX: 'HP',
  Gia: 20000
},
{
  MaSP: 'SP006',
  TenNSX: 'Asus',
  Gia: 18000
},
{
  MaSP: 'SP008',
  TenNSX: 'Microsoft',
  Gia: 25000
}
]

```

C) Thực hiện truy vấn MongoDB với các yêu cầu sau:

1) Xóa thông tin của tất cả các nhà sản xuất mà không sản xuất bất kỳ một sản phẩm PC nào.

```

TH3> // B1: Lấy danh sách các MaNSX có sản xuất PC (Loại bỏ trùng lặp bằng distinct)
let danhSachNsxCoPC = db.sanpham.distinct("MaNSX", { Loai: "PC" });

// B2: Xóa các NSX có _id KHÔNG nằm trong ($nin) danh sách trên
db.nsx.deleteMany({ _id: { $nin: danhSachNsxCoPC } });

print("Đã xóa xong NSX không có PC!");
Đã xóa xong NSX không có PC!

TH3> |

```

Ảnh minh họa truy vấn xóa các nhà sản xuất không sản xuất bất kỳ máy PC nào. Truy vấn này yêu cầu kiểm tra điều kiện tồn tại dữ liệu liên quan, thể hiện khả năng xử lý logic phức tạp trong MongoDB.

2) Tìm tên của các nhà sản xuất chỉ sản xuất Laptop mà không sản xuất PC.

```

TH3> // Lấy danh sách MaNSX
let nsxCoLaptop = db.sanpham.distinct("MaNSX", { Loai: "Laptop" });
let nsxCoPC = db.sanpham.distinct("MaNSX", { Loai: "PC" });

// Tìm và in ra Tên NSX
db.nsx.find(
  {
    _id: {
      $in: nsxCoLaptop, // Có sản xuất Laptop
      $nin: nsxCoPC     // KHÔNG sản xuất PC
    }
  },
  { TenNSX: 1, _id: 0 } // Chỉ hiển thị cột TenNSX
);

[
  { TenNSX: 'Apple' },
  { TenNSX: 'Lenovo' },
  { TenNSX: 'Acer' },
  { TenNSX: 'Razer' },
  { TenNSX: 'MSI' }
]
TH3> |

```

Ảnh thể hiện truy vấn tìm các nhà sản xuất chỉ sản xuất Laptop mà không sản xuất PC. Điều này yêu cầu lọc dữ liệu theo nhiều điều kiện, thể hiện khả năng truy vấn linh hoạt của MongoDB

3) Cập nhật giá cho tất cả các máy PC của nhà sản xuất tên là Dell tăng lên 15%


```
TH3> // B1: Tìm ID của hãng Dell
let idDell = db.nsx.findOne({ TenNSX: "Dell" })._id;

// B2: Tìm ID của các sản phẩm PC thuộc hãng Dell
let danhSachPcDell = db.sanpham.distinct("_id", { MaNSX: idDell, Loai: "PC" });

// =====
// B3: IN RA DỮ LIỆU TRƯỚC KHI CẬP NHẬT
// =====
print("\n[1] --- GIÁ TRƯỚC KHI TĂNG 15% ---");
let truoocCapNhat = db.pc.find(
  { _id: { $in: danhSachPcDell } },
  { _id: 1, CPU: 1, Gia: 1 } // Chỉ in mã máy, CPU và Giá cho dễ nhìn
).toArray();
printjson(truoocCapNhat);

// =====
// B4: THỰC HIỆN CẬP NHẬT TĂNG 15%
// =====
db.pc.updateMany(
  { _id: { $in: danhSachPcDell } },
  { $mul: { Gia: 1.15 } }
);
print("\n>>> Đã chạy lệnh cập nhật tăng 15% thành công! <<<");

// =====
// B5: IN RA DỮ LIỆU SAU KHI CẬP NHẬT ĐỂ KIỂM TRA
// =====
print("\n[2] --- GIÁ SAU KHI TĂNG 15% ---");
let sauCapNhat = db.pc.find(
  { _id: { $in: danhSachPcDell } },
  { _id: 1, CPU: 1, Gia: 1 }
).toArray();
printjson(sauCapNhat);
print("\n");
```

```
[1] --- GIÁ TRƯỚC KHI TĂNG 15% ---
[
  {
    _id: 'SP001',
    CPU: 'Intel i5-10400',
    Gia: 19837.5
  }
]

>>> Đã chạy lệnh cập nhật tăng 15% thành công! <<<

[2] --- GIÁ SAU KHI TĂNG 15% ---
[
  {
    _id: 'SP001',
    CPU: 'Intel i5-10400',
    Gia: 22813.125
  }
]
```

Ảnh minh họa thao tác cập nhật giá sản phẩm PC của một nhà sản xuất (ví dụ: Dell) tăng 15%. Truy vấn sử dụng toán tử cập nhật (\$mul hoặc \$set), cho thấy MongoDB hỗ trợ cập nhật hàng loạt dữ liệu hiệu quả.

4) Tìm tên của các nhà sản xuất có ít nhất 2 máy tính thuộc loại khác nhau (PC và Laptop) có cùng tốc độ CPU.

```
TH3> db.sanpham.aggregate([
  // Nối với bảng pc và laptop để lấy cấu hình CPU
  { $lookup: { from: "pc", localField: "_id", foreignField: "_id", as: "pc_info" } },
  { $lookup: { from: "laptop", localField: "_id", foreignField: "_id", as: "laptop_info" } },

  // Tạo một trường CPU chung (lấy từ PC hoặc Laptop)
  { $project: {
    MaNSX: 1,
    Loai: 1,
    cpu: { $ifNull: [ { $arrayElemAt: [ "$pc_info.CPU", 0 ] }, { $arrayElemAt: [ "$laptop_info.CPU", 0 ] } ] }
  } },

  // Gom nhóm theo Mã NSX và tên CPU
  { $group: {
    _id: { MaNSX: "$MaNSX", cpu: "$cpu" },
    loaiSanPham: { $addToSet: "$Loai" } // Gom các loại sản phẩm vào một mảng
  } },

  // Lọc ra nhóm nào mà mảng loaiSanPham chứa CẢ "PC" và "Laptop"
  { $match: {
    loaiSanPham: { $all: [ "PC", "Laptop" ] }
  } },

  // Nối bảng nsx để lấy Tên in ra màn hình
  { $lookup: { from: "nsx", localField: "_id.MaNSX", foreignField: "_id", as: "nsx_info" } },
  { $unwind: "$nsx_info" },
  { $project: { TenNSX: "$nsx_info.TenNSX", "CPU_GiongNhuu": "$_id.cpu", _id: 0 } }
]);
```

Ảnh thể hiện truy vấn tìm các nhà sản xuất có cả PC và Laptop với cùng tốc độ CPU. Đây là truy vấn nâng cao, yêu cầu so sánh dữ liệu giữa các collection khác nhau, thể hiện khả năng xử lý logic phức tạp.

5) Tìm tên của các nhà sản xuất của tất cả các sản phẩm (PC hoặc Laptop) có dung lượng ổ cứng tối thiểu 128 GB.


```

TH3> // B1: Tìm _id các sản phẩm thỏa mãn điều kiện ổ cứng
| let spHopLe = [];
| db.pc.find({ HD: /128GB|256GB|512GB|TB/ }).forEach(doc => spHopLe.push(doc._id));
| db.laptop.find({ HD: /128GB|256GB|512GB|TB/ }).forEach(doc => spHopLe.push(doc._id));
|
| // B2: Lấy danh sách MaNSX từ bảng sanpham
| let dsNsxId = db.sanpham.distinct("MaNSX", { _id: { $in: spHopLe } });
|
| // B3: In ra tên NSX
| db.nsx.find(
|   { _id: { $in: dsNsxId } },
|   { TenNSX: 1, _id: 0 }
| );
|
| [
|   { TenNSX: 'Dell' },
|   { TenNSX: 'HP' },
|   { TenNSX: 'Apple' },
|   { TenNSX: 'Lenovo' },
|   { TenNSX: 'Asus' },
|   { TenNSX: 'Acer' },
|   { TenNSX: 'Microsoft' },
|   { TenNSX: 'Razer' },
|   { TenNSX: 'MSI' }
| ]
TH3> |

```

Ảnh minh họa truy vấn tìm các nhà sản xuất có sản phẩm với dung lượng ổ cứng tối thiểu 128GB. Truy vấn sử dụng toán tử so sánh (\$gte) để lọc dữ liệu theo điều kiện.

6) Tìm tên của các nhà sản xuất của máy PC có dung lượng ổ cứng lớn nhất.

```

TH3> let maxHD_GB = 0;
| let pcIdLonNhat = [];
|
| // Duyệt qua từng máy PC để phân tích dung lượng
| db.pc.find().forEach(may_pc => {
|   let dungLuong = may_pc.HD; // VD: "1TB HDD"
|   let soGB = 0;
|
|   // Đổi TB ra GB (1TB = 1024)
|   if (dungLuong.includes("TB")) {
|     soGB = parseInt(dungLuong) * 1024;
|   } else {
|     soGB = parseInt(dungLuong);
|   }
|
|   // Cập nhật giá trị MAX
|   if (soGB > maxHD_GB) {
|     maxHD_GB = soGB;
|     pcIdLonNhat = [may_pc._id];
|   } else if (soGB === maxHD_GB) {
|     pcIdLonNhat.push(may_pc._id);
|   }
| });
|
| // Tìm MaNSX và in ra tên
| let nsxId_MAX = db.sanpham.distinct("MaNSX", { _id: { $in: pcIdLonNhat } });
| print("Tên NSX có PC ổ cứng lớn nhất là:");
| db.nsx.find({ _id: { $in: nsxId_MAX } }, { TenNSX: 1, _id: 0 });
|
| Tên NSX có PC ổ cứng lớn nhất là:
| [ { TenNSX: 'Microsoft' } ]
TH3> |

```

Ảnh thể hiện truy vấn tìm nhà sản xuất có sản phẩm PC sở hữu dung lượng ổ cứng lớn nhất. Thường sử dụng sort kết hợp limit để lấy giá trị lớn nhất.

D) Kiểm tra hiệu năng truy vấn trong MongoDB

Thực hiện các cách để đo lường và cải thiện hiệu năng truy vấn:

1) Dùng .explain() để phân tích truy vấn:

Gắn .explain("executionStats") vào truy vấn để xem chi tiết về cách MongoDB thực hiện.

Kết quả gồm:

- executionTimeMillis: thời gian chạy truy vấn (ms)
- totalDocsExamined: số document MongoDB đã đọc
- nReturned: số document được trả về
- indexUsed: có sử dụng index hay không

```
// 2. Chạy thử một truy vấn và dùng .explain() để phân tích chi tiết
print("\n--- PHÂN TÍCH TRUY VẤN TÌM PC BẰNG EXPLAIN ---");
let ketQuaExplain = db.sanpham.find({ Loai: "PC" }).explain("executionStats");

print("- Thời gian chạy (ms):", ketQuaExplain.executionStats.executionTimeMillis);
print("- Số document trả về (nReturned):", ketQuaExplain.executionStats.nReturned);
print("- Số document đã phải quét (totalDocsExamined):", ketQuaExplain.executionStats.totalDocsExamined);

// Kiểm tra xem có dùng Index không (Nếu là COLLSCAN tức là quét toàn bảng, IXSCAN là dùng Index)
let dungIndex = ketQuaExplain.queryPlanner.winningPlan.stage;
print("- Cách quét dữ liệu:", dungIndex === "COLLSCAN" ? "Quét toàn bảng (Chưa tối ưu)" : "Dùng Index (Đã tối ưu)");
```

```
--- PHÂN TÍCH TRUY VẤN TÌM PC BẰNG EXPLAIN ---
- Thời gian chạy (ms): 0
- Số document trả về (nReturned): 4
- Số document đã phải quét (totalDocsExamined): 10
- Cách quét dữ liệu: Quét toàn bảng (Chưa tối ưu)
```

Ảnh minh họa việc sử dụng .explain("executionStats") để phân tích cách MongoDB thực thi truy vấn. Công cụ này cung cấp thông tin chi tiết về thời gian chạy, số document đã đọc và việc sử dụng index, từ đó giúp đánh giá hiệu năng truy vấn.

2) Xem các truy vấn chậm (slow queries):

MongoDB lưu lại các truy vấn chạy chậm hơn slowOpThresholdMs (mặc định 100ms) trong log.

Cách thực hiện:

- Bật profiler

```
TH3> // 1. Bật Profiler: Cài đặt mức 1 (Chỉ ghi log các truy vấn chậm hơn 100ms)
| db.setProfilingLevel(1, { slowms: 100 });
| print("Đã bật Profiler (Mức 1, ngưỡng 100ms).");
```

```
Đã bật Profiler (Mức 1, ngưỡng 100ms).
```

Ảnh thể hiện thao tác bật profiler để ghi lại các truy vấn. Khi bật, MongoDB sẽ lưu thông tin truy vấn vào system.profile, giúp theo dõi và phân tích các truy vấn chậm hoặc không tối ưu.

- Xem kết quả trong hệ thống log hoặc collection system.profile

```

TH3> db.setProfilingLevel(2)
|
| // chạy query
| db.users.find({ age: { $gt: 20 } })
|
| // rồi mới xem
| db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
|
| {
|   op: 'query',
|   ns: 'TH3.system.profile',
|   command: {
|     find: 'system.profile',
|     filter: {},
|     sort: { ts: -1 },
|     limit: 5,
|     lsid: { id: UUID('f4fb44e8-c429-40f5-92b4-fc61951b675f') },
|     '$db': 'TH3'
|   },
|   keysExamined: 0,
|   docsExamined: 0,
|   hasSortStage: true,
|   nBatches: 1,
|   cursorExhausted: true,
|   numYield: 0,
|   nreturned: 0,
|   planCacheShapeHash: '9092A9A1',
|   queryHash: '9092A9A1',
|   planCacheKey: 'D67F070E',
|   queryFramework: 'classic',
|   locks: { Global: { acquireCount: { r: Long('1') } } },
|   flowControl: {},
|   responseLength: 107,
|   protocol: 'op_msg',
|   millis: 0,
|   planSummary: 'COLLSCAN',
|   planningTimeMicros: 183,
|
|   execStats: {
|     isCached: false,
|     stage: 'SORT',
|     nReturned: 0,
|     executionTimeMillisEstimate: 0,
|     works: 2,
|     advanced: 0,
|     needTime: 1,
|     needYield: 0,
|     saveState: 0,
|     restoreState: 0,
|     isEOF: 1,
|     sortPattern: { ts: -1 },
|     memLimit: 104857600,
|     limitAmount: 5,
|     type: 'simple',
|     totalDataSizeSorted: 0,
|     usedDisk: false,
|     spills: 0,
|     spilledRecords: 0,
|     spilledBytes: 0,
|     spilledDataStorageSize: 0,
|     inputStage: {
|       stage: 'COLLSCAN',
|       nReturned: 0,
|       executionTimeMillisEstimate: 0,
|       works: 1,
|       advanced: 0,
|       needTime: 0,
|       needYield: 0,
|       saveState: 0,
|       restoreState: 0,
|       isEOF: 1,
|       direction: 'forward',
|       docsExamined: 0
|     }
|   },
|   ts: ISODate('2026-04-09T07:04:12.760Z'),
|   client: '127.0.0.1',
|   appName: 'mongosh 2.8.2',
|   allUsers: [],
|   user: ''
| }

```

Ảnh minh họa lệnh truy vấn collection system.profile để xem các truy vấn đã được ghi lại. Đây là nguồn dữ liệu quan trọng để phân tích hiệu năng thực tế của hệ thống; Ảnh thể hiện kết quả profiler với các trường như millis, docsExamined, nreturned. Việc so sánh các giá trị này giúp xác định truy vấn có hiệu quả hay không. Nếu số document quét quá lớn so với kết quả trả về, truy vấn cần được tối ưu.

3) Tối ưu hóa truy vấn:

- **Tạo chỉ mục (Index):** giúp truy vấn nhanh hơn
- **Chỉ lấy các trường cần thiết:** tránh lấy dư dữ liệu
- **Tránh dùng regex, \$where, hoặc truy vấn không dùng index:** các truy vấn này rất chậm khi collection lớn

```
print("\n=====SO SÁNH HIỆU NĂNG TÌM KIẾM MÁY PC GIÁ > 15000");
print("\n=====");

// --- TRƯỚC KHI TỐI ƯU ---
print("[1] TRƯỚC KHI TỐI ƯU (CHƯA CÓ INDEX)");
let start1 = Date.now();
let before = db.pc.find( { gia: { $gt: 15000 } } ).explain("executionStats");
let end1 = Date.now();

print("- Thời gian lệnh Date.now():", (end1 - start1), "ms");
print("- Thời gian hệ thống báo cáo (executionTimeMillis):", before.executionStats.executionTimeMillis, "ms");
// Vì chưa có Index, máy chủ phải lật tung toàn bộ bảng PC lên để kiểm tra giá từng cái
print("- Số document phải quét (totalDocsExamined):", before.executionStats.totalDocsExamined);

// --- TỐI ƯU HÓA ---
print("\n...Đang tạo Index cho cột Giá và áp dụng quy tắc 'Chỉ lấy trường cần thiết'...");
// 1. Tạo Index cho Giá
db.pc.createIndex( { gia: 1 } );

// --- SAU KHI TỐI ƯU ---
print("[2] SAU KHI TỐI ƯU (CÓ INDEX + RÚT GỌN TRƯỜNG DỮ LIỆU)");
let start2 = Date.now();
// 2. Chỉ lấy trường cần thiết (Object: { _id, chỉ lấy CPU và Giá })
let after = db.pc.find(
  { gia: { $gt: 15000 } },
  { _id: 0, CPU: 1, gia: 1 }
).explain("executionStats");
let end2 = Date.now();

print("- Thời gian lệnh Date.now():", (end2 - start2), "ms");
print("- Thời gian hệ thống báo cáo (executionTimeMillis):", after.executionStats.executionTimeMillis, "ms");
// Nhờ có Index (Giống như mục lục sách), máy chủ nhay thẳng đến đúng các document thỏa mãn điều kiện
print("- Số document phải quét (totalDocsExamined):", after.executionStats.totalDocsExamined);
```

```
=====
SO SÁNH HIỆU NĂNG TÌM KIẾM MÁY PC GIÁ > 15000
=====

[1] TRƯỚC KHI TỐI ƯU (CHƯA CÓ INDEX)
- Thời gian lệnh Date.now(): 4 ms
- Thời gian hệ thống báo cáo (executionTimeMillis): 0 ms
- Số document phải quét (totalDocsExamined): 4

...Đang tạo Index cho cột Giá và áp dụng quy tắc 'Chỉ lấy trường cần thiết'...

[2] SAU KHI TỐI ƯU (CÓ INDEX + RÚT GỌN TRƯỜNG DỮ LIỆU)
- Thời gian lệnh Date.now(): 23 ms
- Thời gian hệ thống báo cáo (executionTimeMillis): 18 ms
- Số document phải quét (totalDocsExamined): 4
```

Ảnh minh họa đoạn code sử dụng MongoDB để đo và so sánh hiệu năng truy vấn tìm kiếm các máy PC có giá lớn hơn 15000. Quá trình được chia thành hai giai đoạn: trước khi tối ưu (chưa có index) và sau khi tối ưu (đã tạo index và rút gọn dữ liệu trả về).

Trong giai đoạn đầu, truy vấn được thực hiện với `.find({ Gia: { $gt: 15000 } })` và kết hợp `.explain("executionStats")` để thu thập thông tin chi tiết về cách MongoDB xử lý. Thời gian thực thi cũng được đo thủ công bằng `Date.now()` nhằm đánh giá tổng thời gian chạy thực tế.

Sau đó, hệ thống tiến hành tối ưu bằng cách:

- Tạo index trên trường Gia (`createIndex({ Gia: 1 })`)
- Áp dụng phép chiếu dữ liệu (projection) để chỉ lấy các trường cần thiết (CPU và Giá)

Cuối cùng, truy vấn được thực hiện lại với các điều kiện tối ưu nhằm so sánh sự khác biệt về hiệu năng.

4) So sánh hiệu năng trước / sau:

- Đo thời gian bằng JavaScript:

```
const start = Date.now();
// chạy truy vấn ở đây
const end = Date.now();
console.log("Truy vấn mất:", end - start, "ms");
```

- Hoặc dùng `.explain("executionStats")` để xem chi tiết cách máy chủ xử lý truy vấn

```
print("- Thời gian lệnh Date.now():", (end1 - start1), "ms");
print("- Thời gian hệ thống báo cáo (executionTimeMillis):", before.executionStats.executionTimeMillis, "ms");
// Vì chưa có Index, máy chủ phải lật tung toàn bộ bảng PC lên để kiểm tra giá từng cái
print("- Số document phải quét (totalDocsExamined):", before.executionStats.totalDocsExamined);

let end2 = Date.now();

print("- Thời gian lệnh Date.now():", (end2 - start2), "ms");
print("- Thời gian hệ thống báo cáo (executionTimeMillis):", after.executionStats.executionTimeMillis, "ms");
// Nhờ có Index (Giống như mục lục sách), máy chủ nhay thẳng đến đúng các document thỏa mãn điều kiện
print("- Số document phải quét (totalDocsExamined):", after.executionStats.totalDocsExamined);
```

Ảnh thể hiện cách đo thời gian thực thi truy vấn bằng JavaScript (Date.now()). Đây là phương pháp đơn giản để so sánh hiệu năng trước và sau khi tối ưu, giúp đánh giá hiệu quả của các cải tiến.

```
[1] TRƯỚC KHI TỐI ƯU (CHƯA CÓ INDEX)
- Thời gian lệnh Date.now(): 4 ms
- Thời gian hệ thống báo cáo (executionTimeMillis): 0 ms
- Số document phải quét (totalDocsExamined): 4

...Đang tạo Index cho cột Giá và áp dụng quy tắc 'Chỉ lấy trường cần thiết'...

[2] SAU KHI TỐI ƯU (CÓ INDEX + RÚT GỌN TRƯỜNG DỮ LIỆU)
- Thời gian lệnh Date.now(): 23 ms
- Thời gian hệ thống báo cáo (executionTimeMillis): 18 ms
- Số document phải quét (totalDocsExamined): 4
```

Ảnh minh họa việc so sánh hiệu năng truy vấn trước và sau khi áp dụng tối ưu (ví dụ: tạo index). Sự cải thiện thể hiện qua việc giảm thời gian chạy và giảm số document phải quét, chứng minh tính hiệu quả của các kỹ thuật tối ưu.